

TreeMap

Aspekt	Technische Details
Interne Datenstruktur	TreeMap speichert die Schlüssel-Wert-Paare in einem Rot-Schwarz-Baum, einem speziellen, selbstbalancierenden binären Suchbaum. Dieser garantiert, dass die Pfadlängen von der Wurzel zu jedem Blatt ungefähr gleich sind, was zu konsistenten $O(\log n)$ -Operationen führt.
Sortierung und Ordnung	Natürliche Ordnung oder Comparator: Standardmäßig wird die natürliche Ordnung der Schlüssel verwendet, d. h. die Schlüssel implementieren das Comparable-Interface. Alternativ kann ein benutzerdefinierter Comparator beim Erstellen der TreeMap übergeben werden. Sortierung: Die Elemente werden in sortierter Reihenfolge der Schlüssel abgelegt, was eine geordnete Iteration ermöglicht.
Operationen und Zeitkomplexität	Einfügen (put): Beim Hinzufügen eines Elements wird der Schlüssel im Baum gesucht ($O(\log n)$). Falls der Schlüssel bereits existiert, wird der Wert ersetzt; andernfalls wird ein neuer Knoten eingefügt, gefolgt von eventuellen Rotationen und Farbänderungen, um die Baum-Balance zu wahren. Suchen (get, containsKey): Finden eines Schlüssels erfolgt ebenfalls in $O(\log n)$ durch Baumdurchquerung. Löschen (remove): Entfernen eines Elements erfordert das Finden des Knotens ($O(\log n)$) und eine anschließende Rebalancierung.
Iterieren und Zugriff	Geordnete Iteration: Ein Iterator liefert die Elemente in der sortierten Reihenfolge, basierend auf der Schlüsselvergleichslogik. NavigableMap-Methoden: Methoden wie <code>firstKey()</code> , <code>lastKey()</code> , <code>ceilingKey()</code> oder <code>floorKey()</code> ermöglichen den Zugriff auf Elemente in Bezug auf ihre natürliche Reihenfolge.
Null-Werte und Schlüssel	Null-Schlüssel: TreeMap erlaubt keine null-Schlüssel, da sie auf der natürlichen Ordnung oder einem Comparator basiert, der null nicht verarbeiten kann. Null-Werte: Die Werte in der Map dürfen jedoch null sein.
Thread-Safety	TreeMap ist nicht synchronisiert. Bei parallelen Zugriffen muss eine externe Synchronisation erfolgen (z. B. über <code>Collections.synchronizedSortedMap</code> oder durch den Einsatz von Concurrent NavigableMap-Implementierungen).
Anwendungsfall im Beispiel	Im Airplane-Beispiel wird eine TreeMap verwendet, um Sitznummern (als Strings) auf einen Belegungsstatus (Boolean) abzubilden. Dabei profitieren wir von: Sortierter Darstellung: Die Sitzplätze werden in einer definierten Reihenfolge abgelegt, was insbesondere beim Durchlaufen (<code>printSeating</code>) hilfreich ist. Effiziente Suche: Die Methoden wie <code>computeIfPresent</code> nutzen die $O(\log n)$ -Zeitkomplexität, um den Status einzelner Sitzplätze schnell zu aktualisieren.